Software Security (KOSEN69) | Week 12

Software Supply-Chain Security

Week 12 assignment report

Student: Wanna San

Student ID: 6631503097

Group: The Imposters

Date: 24 October 2026

Technical evidence complete | Student review and Git publication pending

Observed technical results are complete. Suggested personal text is clearly labeled and requires student review. Branch push, pull request, quiz and course-portal submission are not represented as completed.

Evidence pages retain complete original screenshots. No authentication secrets are included.

1. Assignment overview and evidence integrity

This report follows Worksheet 12 Parts 1-4 and the required Audit the AI, Explain in Plain English (EiPE), and Prompt Problem. The supplied Python application and dependency pins were preserved. Work took place on branch wk12 in /private/tmp/software-security-wk12, starting from upstream/main eff680baf8d5f85e6cd736da776fb9d40e61c043; the unrelated wk06 working tree was not modified.

Student information	Recorded value
Name / ID	Wanna San / 6631503097
Group / date	The Imposters / 24 October 2026
AI disclosure	Codex inspected files, ran authorized commands, arranged captures, drafted explanations and produced this PDF. Student personally selected browser presets and completed GitHub OIDC/2FA.
Personal text status	Suggested EiPE, critique and reflection are labeled for student review; no invented personal experience is presented.
Flag	Not issued in the repository evidence; conditional worksheet field is not applicable unless the instructor supplies one.

Observed results come from saved tool output, immutable digests and genuine screenshots. Technical commands were executed by Codex with explicit authorization; descriptions do not claim the student manually ran them. Simulations, read-only checks and actual registry/signing operations are identified separately. Earlier chronological notes are preserved in supporting/week12-working-history.md; this document gives the final state.

Every embedded screenshot is an unmodified, full-display PNG with the terminal identity command visible in the same image. Landscape A3 evidence pages preserve the complete screen without cropping; zoom to inspect dense terminal output. Rejected captures remain on disk but are excluded from the PDF and submission commits. Long commands may wrap visually; saved command JSON and the Markdown report preserve their exact text.

```
printf '%s | %s | ' "$(whoami)" '6631503097'; date '+%F %T %Z'
```

Environment / tool	Observed version
Host / Docker	macOS 27.0.1 ARM64; Docker Desktop 4.73.0 / Engine 29.4.3, Linux ARM64
Git / Bash / Python	Git 2.54.0; Bash 3.2.57; host Python 3.13.2
SCA tools	Trivy 0.74.0; pip-audit 2.9.0 (version rechecked immediately after scan)
Signing / pip refusal	Official Cosign v3.1.4 darwin/arm64, checksum and existing release signature verified; disposable venv pip 25.0

2. Part 2 - lecture answers

Q1. Dependency confusion

In merged-index mode, a resolver considers compatible candidates from both sources; it does not treat the private index as authoritative. In the simulation, public 99.0.0 wins over private 1.4.0. A higher compatible public version can therefore substitute an internal dependency. Important correction to the simplified question: an exact requirement ==1.4.0 excludes 99.0.0; the example concerns two available versions that the requirement permits, not bypass of an exact pin. A pin alone still does not authenticate the bytes or source of a same-version artifact. [R1]

Q2. Typosquatting and install-time execution

A misspelled name such as reqeusts can select an unrelated attacker-controlled distribution. Source-distribution build backends or legacy setup.py steps can execute during build/install before application imports, so avoiding an import does not eliminate that risk. This is a possibility, not a claim that every wheel executes setup.py: standard wheel installation has a different mechanism. Use vetted names and sources, reviewed hashes and isolated builds; never install the illustrative look-alikes. [R1, R4]

Q3. SBOM, CycloneDX and SPDX

An SBOM is a structured inventory of components, versions, identifiers and relationships. CycloneDX and SPDX provide interoperable formats for that information. It lets responders identify potentially affected artifacts when a dependency advisory appears; this report binds the inventory to the image digest. The worksheet calls it a prerequisite for provenance: it is useful supporting inventory, but an SBOM is not itself build provenance and is not a universal formal prerequisite for a SLSA level. Provenance must describe the build and its inputs. [R5, R6, R8]

Q4. Keyless signing: Sigstore, Fulcio, Rekor and OIDC

OIDC authenticates the signer; Fulcio issues a short-lived certificate binding an ephemeral key to that identity. Rekor records public signing evidence for auditability. This reduces exposure from a reusable long-lived private key, including hard-coded-key risk (CWE-321), but does not remove account compromise or malicious-code risk. Our approved GitHub signing disclosed the confirmed email, and exact-policy verification required that email plus the GitHub OAuth issuer. Registry authentication and OIDC authentication are separate. [R9, R10]

Q5. SLSA levels and the evidence-based assessment

The approved SLSA v1.2 Build track has L0 (no guarantees), L1 (build provenance exists), L2 (authenticated provenance generated by a hosted build platform), and L3 (a hardened build platform resisting tampering). The historical unnamed 1-4 model is not the current Build-track model. A signature, SBOM and a gate do not automatically establish a level: provenance contents and platform guarantees must be assessed. We claim no SLSA level, because the verified statement is an artifact-signature predicate and no complete trusted-builder provenance policy was evaluated. [R7, R8]

3. Task 0 - nine intentionally outdated dependencies

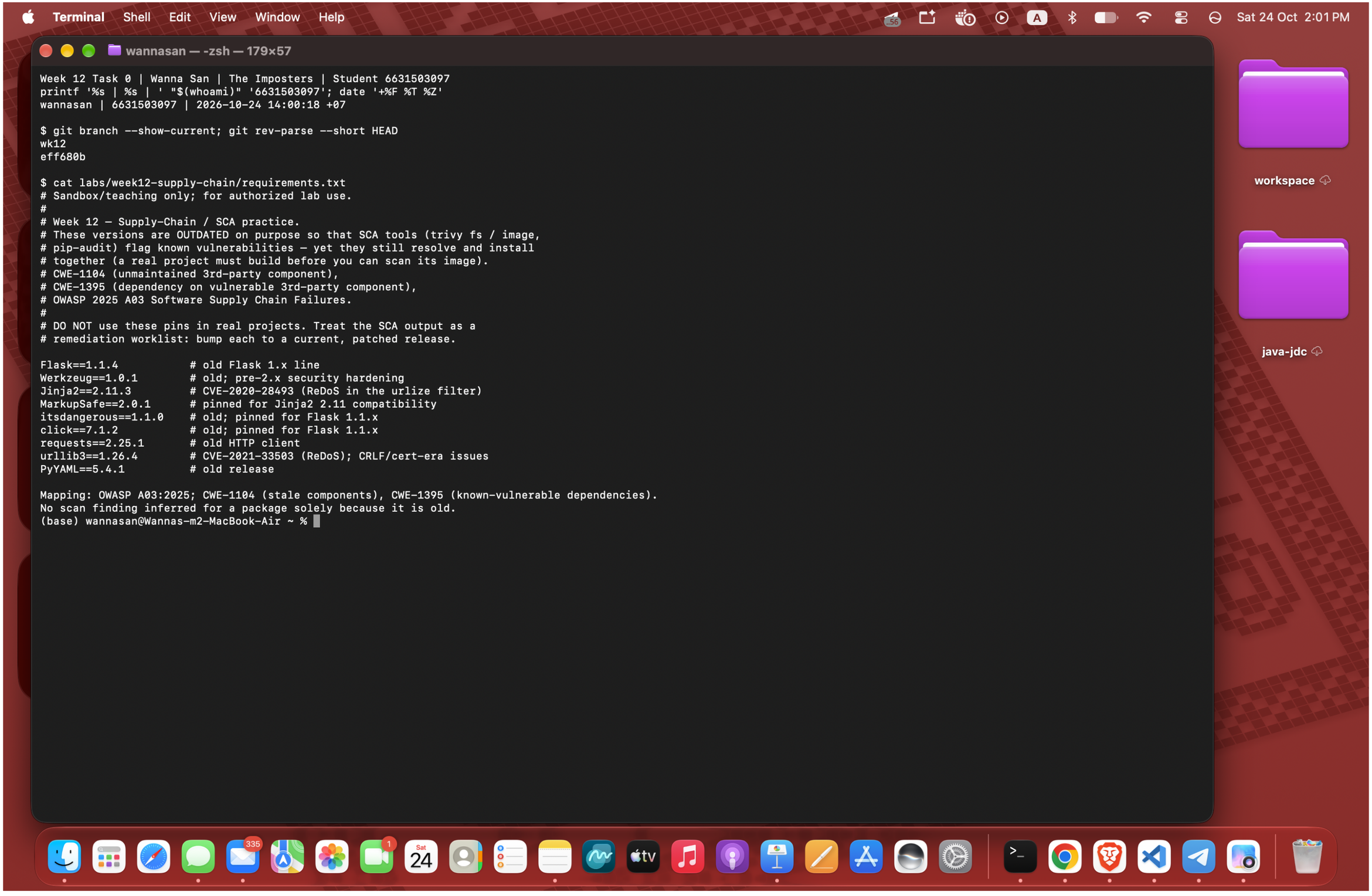
Observed command: cat labs/week12-supply-chain/requirements.txt. The exact instructor manifest is retained in evidence/task0/requirements-original.txt. The final evidence audit re-parsed all nine pins and confirmed the instructor file hash was unchanged. Browser/terminal inventory reading has no scanner exit status; the final structured inventory check completed successfully.

Package	Pinned version	Mapping
Flask	1.1.4	A03; CWE-1395 when a vulnerability is confirmed
Werkzeug	1.0.1	A03; CWE-1395 when a vulnerability is confirmed
Jinja2	2.11.3	A03; CWE-1395 when a vulnerability is confirmed
MarkupSafe	2.0.1	A03; CWE-1395 when a vulnerability is confirmed
itsdangerous	1.1.0	A03; CWE-1395 when a vulnerability is confirmed
click	7.1.2	A03; CWE-1395 when a vulnerability is confirmed
requests	2.25.1	A03; CWE-1395 when a vulnerability is confirmed
urllib3	1.26.4	A03; CWE-1395 when a vulnerability is confirmed
PyYAML	5.4.1	A03; CWE-1395 when a vulnerability is confirmed

These versions were intentionally selected for SCA training. Age alone does not prove every pin is unmaintained or vulnerable; CWE-1104 applies when lack of maintenance is established, while confirmed vulnerable components fit CWE-1395. The scan results below, not manifest comments alone, establish the recorded findings. [R2, R3]

Remediation note: select mutually compatible patched releases based on the complete advisory worklist, then test the resolved tree and review hashes. Keep the intentionally vulnerable instructor manifest intact; proposed upgrades here were not implemented or claimed to pass a re-scan.

Task 0: original nine-package manifest, wk12 branch and exact identity command. Genuine full-screen inventory evidence.



evidence/screenshots/task0.png | Original full-screen PNG; see source for full resolution.

4. Task 1 - SCA findings and remediation worklist

```
cd labs/week12-supply-chain
bash sca_scan.sh
```

The instructor script ran unchanged. A student-owned Docker wrapper preserved every subprocess argument, stdout, stderr and real exit status, because the script masks statuses with || true. Complete output is in evidence/task1/; command records are in process-status.jsonl.

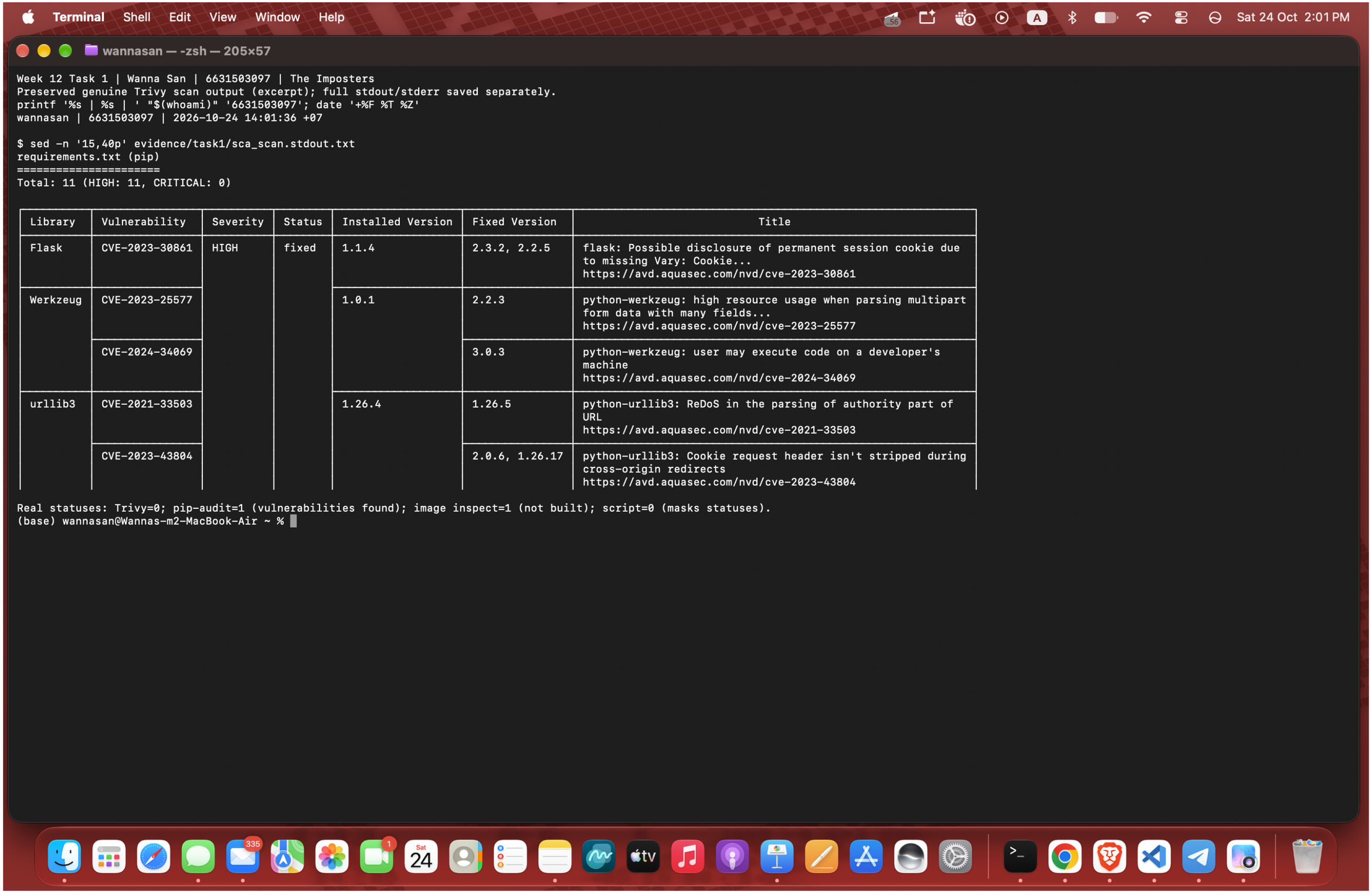
Operation	Actual result / exit
Trivy fs, HIGH/CRITICAL	11 HIGH, 0 CRITICAL; exit 0 (no vulnerability-failure exit policy configured)
pip-audit -r requirements.txt	33 known vulnerabilities in six packages; exit 1 because findings were present
Optional image inspection	Exit 1 at scan time because the lab image was not yet built; image scan skipped
Whole instructor script	Exit 0, which does not by itself establish scanner success

Package	Installed	Observed CVE	Severity	Advisory-specific fixed versions
Flask	1.1.4	CVE-2023-30861	HIGH	2.3.2, 2.2.5
Werkzeug	1.0.1	CVE-2023-25577	HIGH	2.2.3
urllib3	1.26.4	CVE-2021-33503	HIGH	1.26.5

The fixed versions above are exactly those in the Trivy table for those individual advisories, not a recommendation that these old releases fix all later findings. pip-audit also detected transitive idna, absent from the nine direct pins, and did not report severity in its table. Trivy warned that license detection was skipped because the filesystem target lacked site-packages. The two tools have different coverage and filters, so their counts need not match.

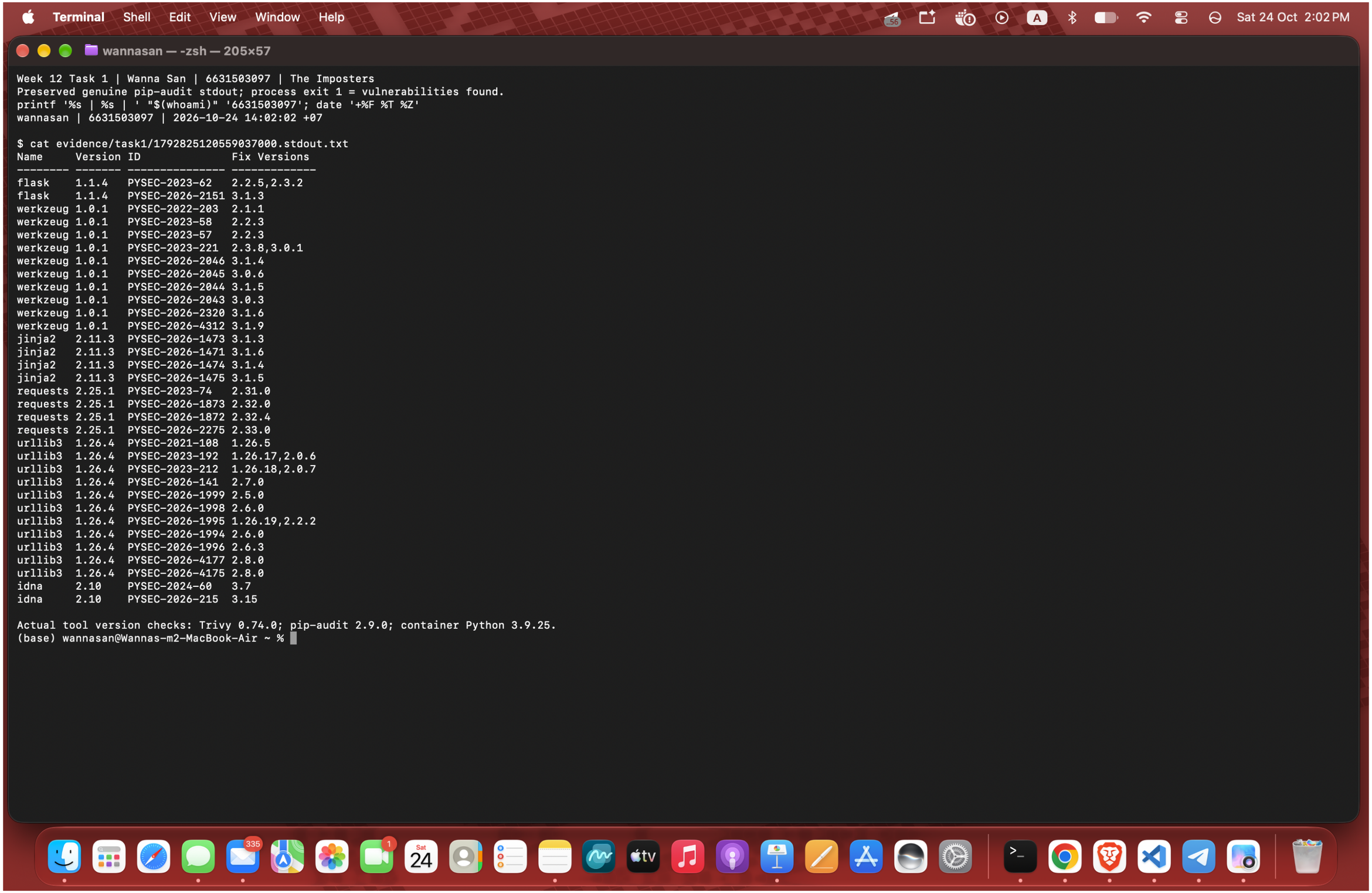
Remediation note (A03/CWE-1395): prioritize reachable/high-impact findings and resolve the complete compatible dependency set, then re-scan and test before claiming remediation. Hashes and signatures bind bytes/identity; they do not patch vulnerable behavior. Full package-upgrade remediation was not performed in this lab. [R2, R3]

Task 1: genuine preserved Trivy output with HIGH findings, installed/fixed versions and identity command. Log replay is explicitly labeled.



evidence/screenshots/task1-trivy.png | Original full-screen PNG; see source for full resolution.

Task 1: genuine preserved pip-audit advisory output, identity command and recorded tool versions; exit 1 denotes findings.



evidence/screenshots/task1-pip-audit.png | Original full-screen PNG; see source for full resolution.

5. Task 2 - Dependency Confusion Heist simulation

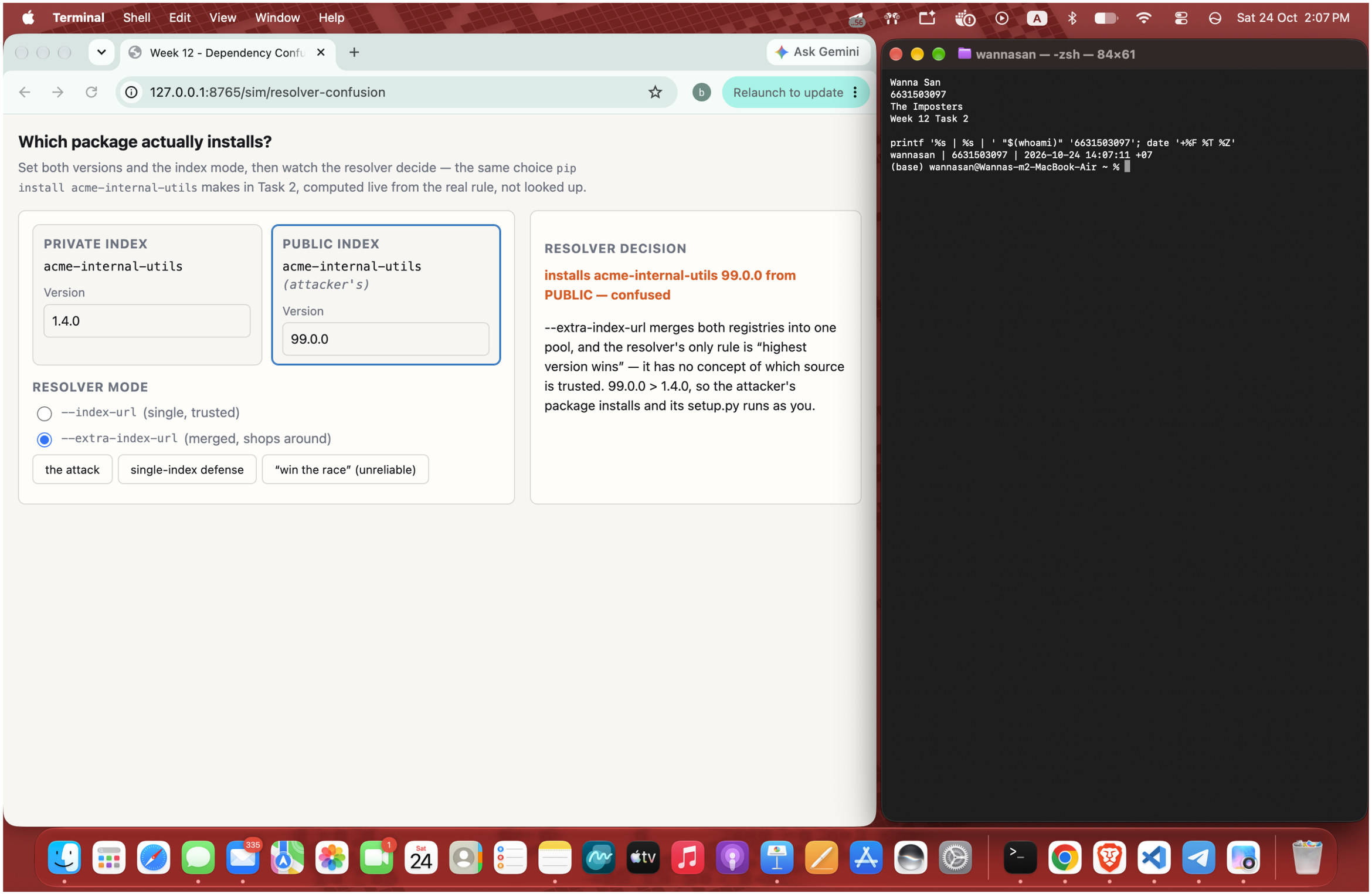
Only the instructor-provided resolver simulation was used for dependency confusion. No look-alike package, marker payload, private/public attack registry or real dependency-confusion install was created. Student manually selected the final browser states; Codex arranged real Chrome and Terminal, executed the identity command and captured the whole display. There is no CLI exit code for a browser preset click.

Preset / mode	Private / public	Observed browser verdict
Attack / merged	1.4.0 / 99.0.0	Public 99.0.0 selected - confused
Same versions / single	1.4.0 / 99.0.0	Private 1.4.0 selected - safe in the simulated model
Win the race / merged	2.0.0 / 1.9.0	Private 2.0.0 selected - safe this time

The attacker controls the public version and can choose a higher one, so an internal version-number race is not a reliable defense. The JavaScript comparator models the dotted numeric examples, not every PEP 440 constraint or source-selection case; source data are preserved in evidence/task2/engine-results.json and the actual browser captures are the required proof.

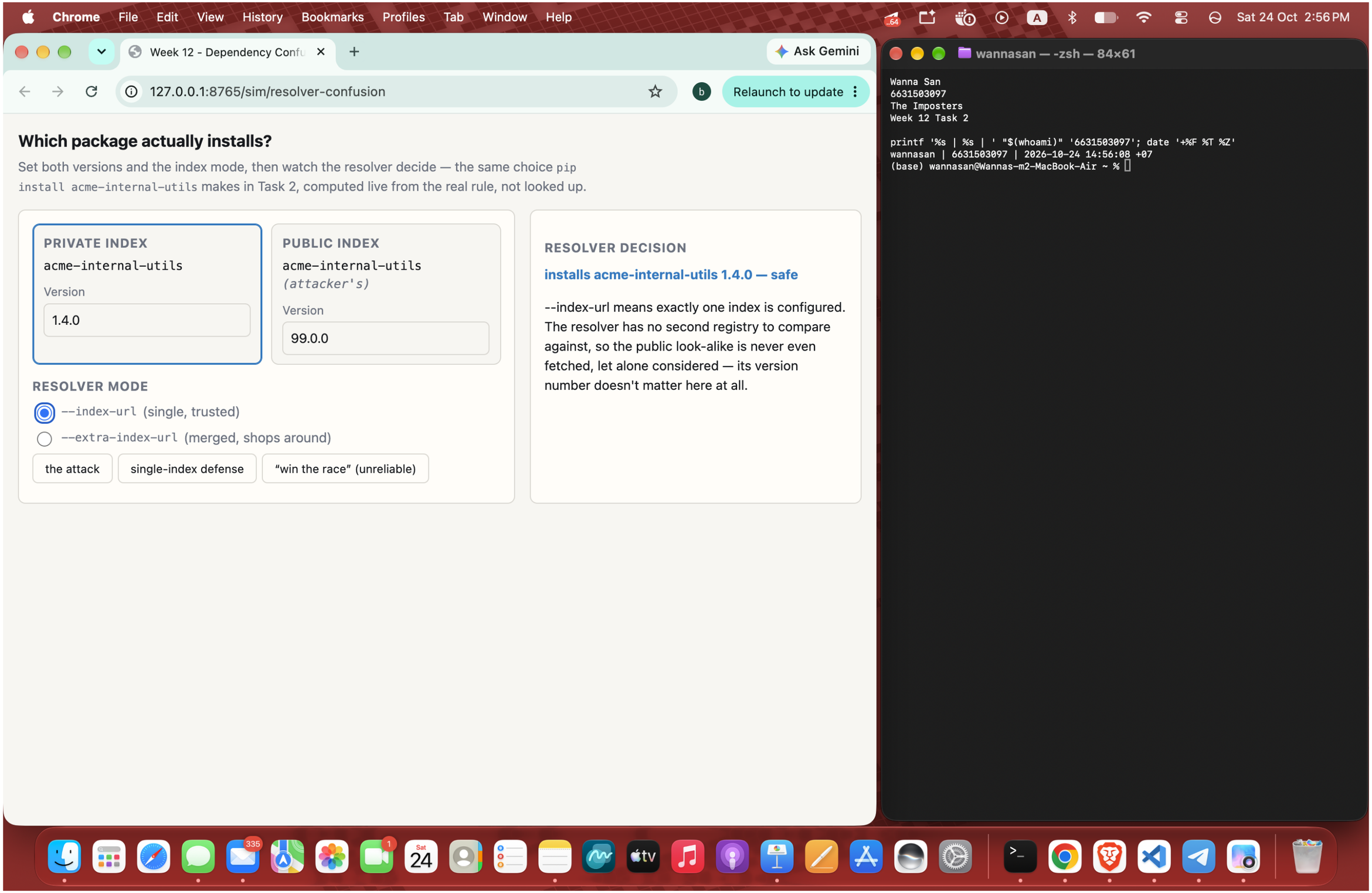
Remediation note (A03/CWE-1357 and CWE-829): one curated trusted source removes the competing public candidate in this model, while reviewed hashes bind acceptable artifacts. Disable extra indexes in environment/configuration as well as CLI, and recognize that compromise of the one trusted source remains a risk. [R1, R4]

Task 2 attack: merged sources, private 1.4.0/public 99.0.0, public version selected. Original PNG; identity proof visible.



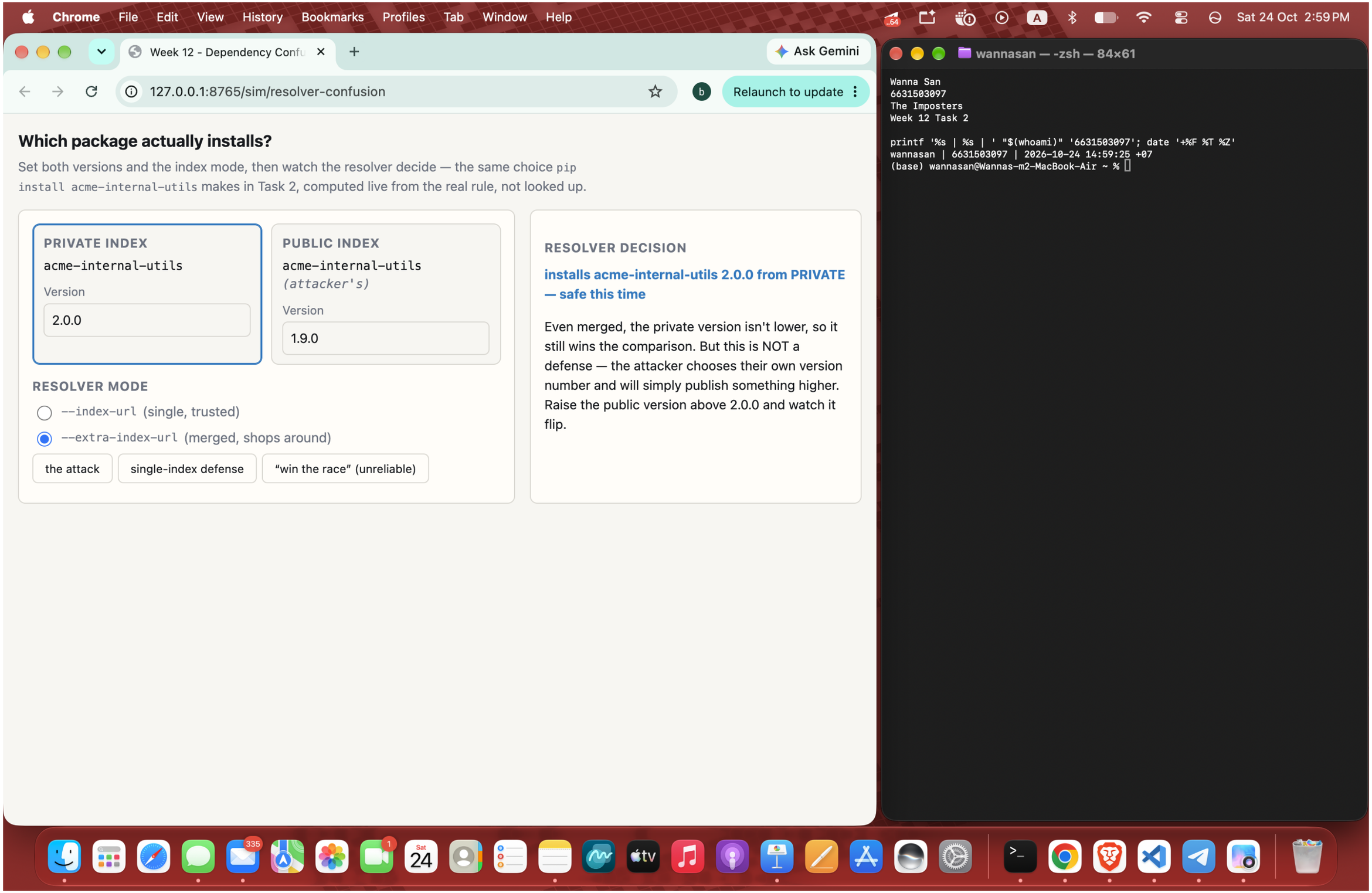
evidence/screenshots/task2-attack-merged-attempt3.png | Original full-screen PNG; see source for full resolution.

Task 2 defense: unchanged version pair with only the trusted index; private 1.4.0 selected. Student manually selected the radio button.



evidence/screenshots/task2-single-index-confirmed.png | Original full-screen PNG; see source for full resolution.

Task 2 unreliable race: merged mode, private 2.0.0/public 1.9.0. The verdict explicitly explains why this is not a durable defense.



evidence/screenshots/task2-race-confirmed.png | Original full-screen PNG; see source for full resolution.

6. Task 3 - image, SBOM, publication and signatures

6.1 Local image build and inventory

```
docker build --progress=plain -t week12-supplychain -t week12-supplychain:lab .
```

Executed from labs/week12-supply-chain. Build exit 0, platform linux/arm64. Local tags week12-supplychain:latest and :lab identified the same artifact. Build stderr includes the pip-as-root warning inside the image and a pip update notice. The image configuration and platform manifest hashes are distinct from the exported OCI index digest; the recorded Docker image ID/index digest below is the one used throughout.

```
sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
```

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v
/private/tmp/software-security-wk12/submission/week12-6631503097/evidence/task3:/evidence
aquasec/trivy:latest image --image-src docker --format cyclonedx --output
/evidence/sbom.cdx.json
sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
```

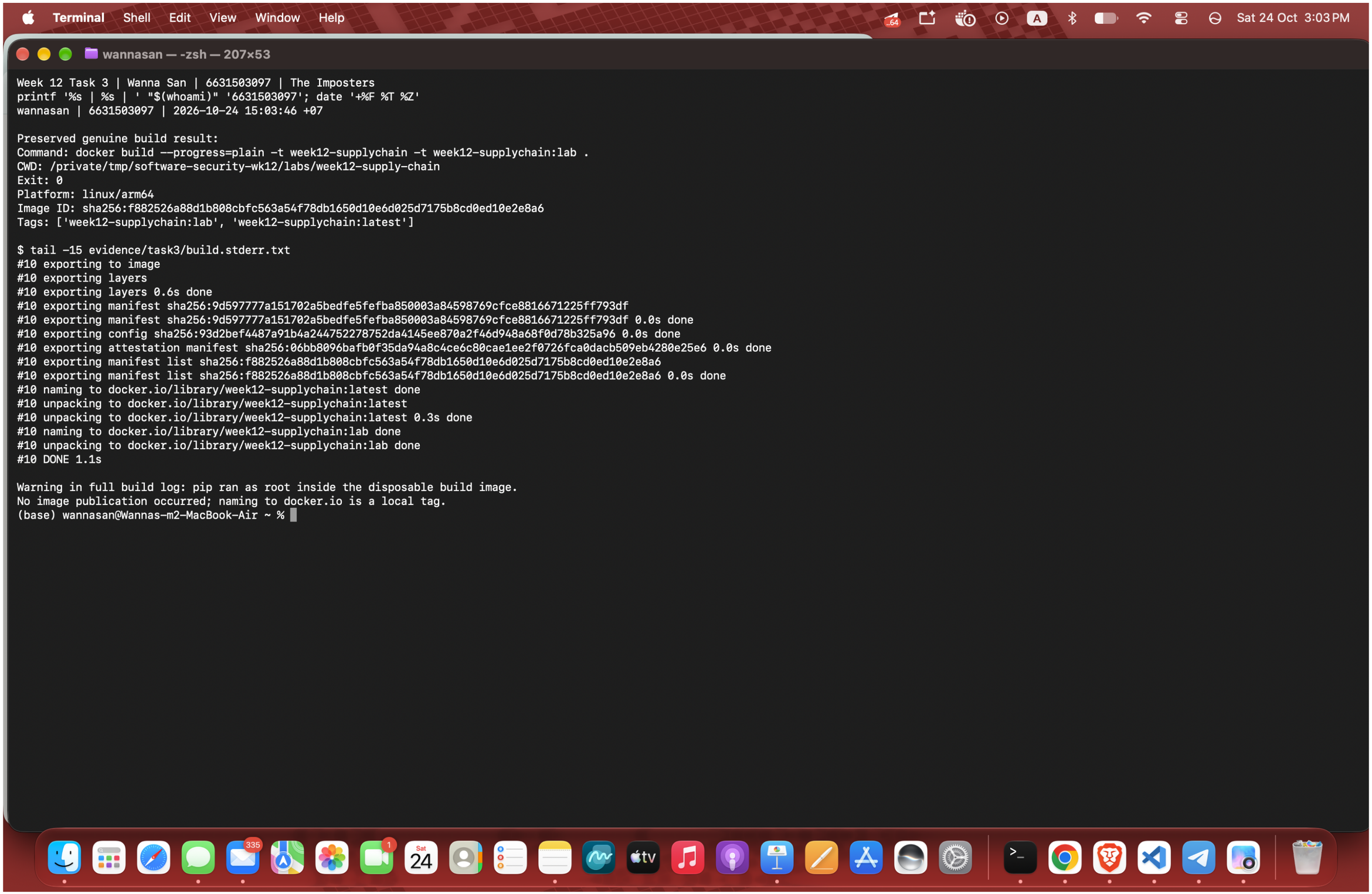
Trivy generated evidence/task3/sbom.cdx.json from the immutable image ID using image --image-src docker, exit 0. CycloneDX 1.7 JSON schema validation passed with zero errors: 119 components (87 Debian packages, 31 Python packages and one OS component), with 120 dependency nodes. The top-level container metadata is separate from that component-array count. This is an image SBOM: it contains OS packages, installed METADATA paths and layer identifiers, and its recorded image ID matches Docker inspect. A network-disabled container read separately confirmed Flask 1.1.4.

Flask field	Actual recorded value
Name / version	Flask / 1.1.4
Type / license	library / BSD-3-Clause
PURL and bom-ref	pkg:pypi/flask@1.1.4
Installed metadata	usr/local/lib/python3.9/site-packages/Flask-1.1.4.dist-info/METADATA
Component SHA-1	caefbe085934d955994ee9bc9d8e4e86cb79c081

The SHA-1 is scanner-reported component metadata, not the image digest or a reviewed pip artifact lock hash. Layer identifiers are retained in flask-component.json. The first schema-host download returned HTTP 403; the official CycloneDX GitHub schema retry passed, and both attempts remain recorded. Trivy noted that CycloneDX output disables vulnerability scanning unless explicitly requested and that license metadata can carry additional terms; this inventory is not a new clean vulnerability scan. [R5]

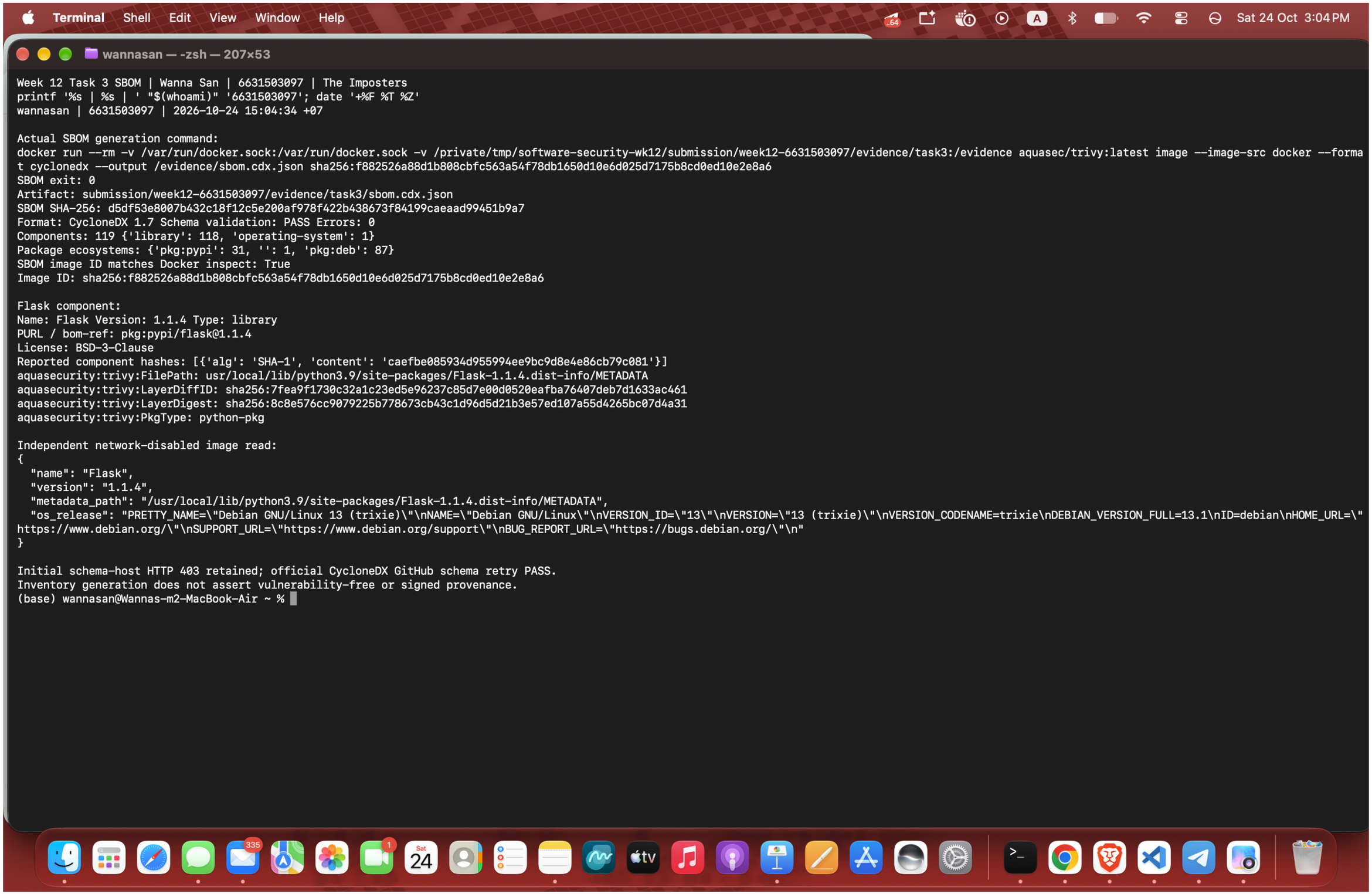
Incident-response note (A03/A08): search this versioned inventory when a new advisory appears, locate affected image digests and prioritize fixes without reconstructing the environment. Include OS and transitive components, preserve the SBOM with the image identity, and do not confuse an inventory with proof of safe behavior or trusted build provenance.

Task 3: real build output, exit 0, linux/arm64 and immutable image ID. Local Docker-Hub-shaped tag names are not evidence of publication.



evidence/screenshots/task3-local-build.png | Original full-screen PNG; see source for full resolution.

Task 3: image-derived CycloneDX inventory, schema PASS, Flask metadata and independent installed-package read, with identity proof.



evidence/screenshots/task3-sbom-validation.png | Original full-screen PNG; see source for full resolution.

6.2 Approved private GHCR publication

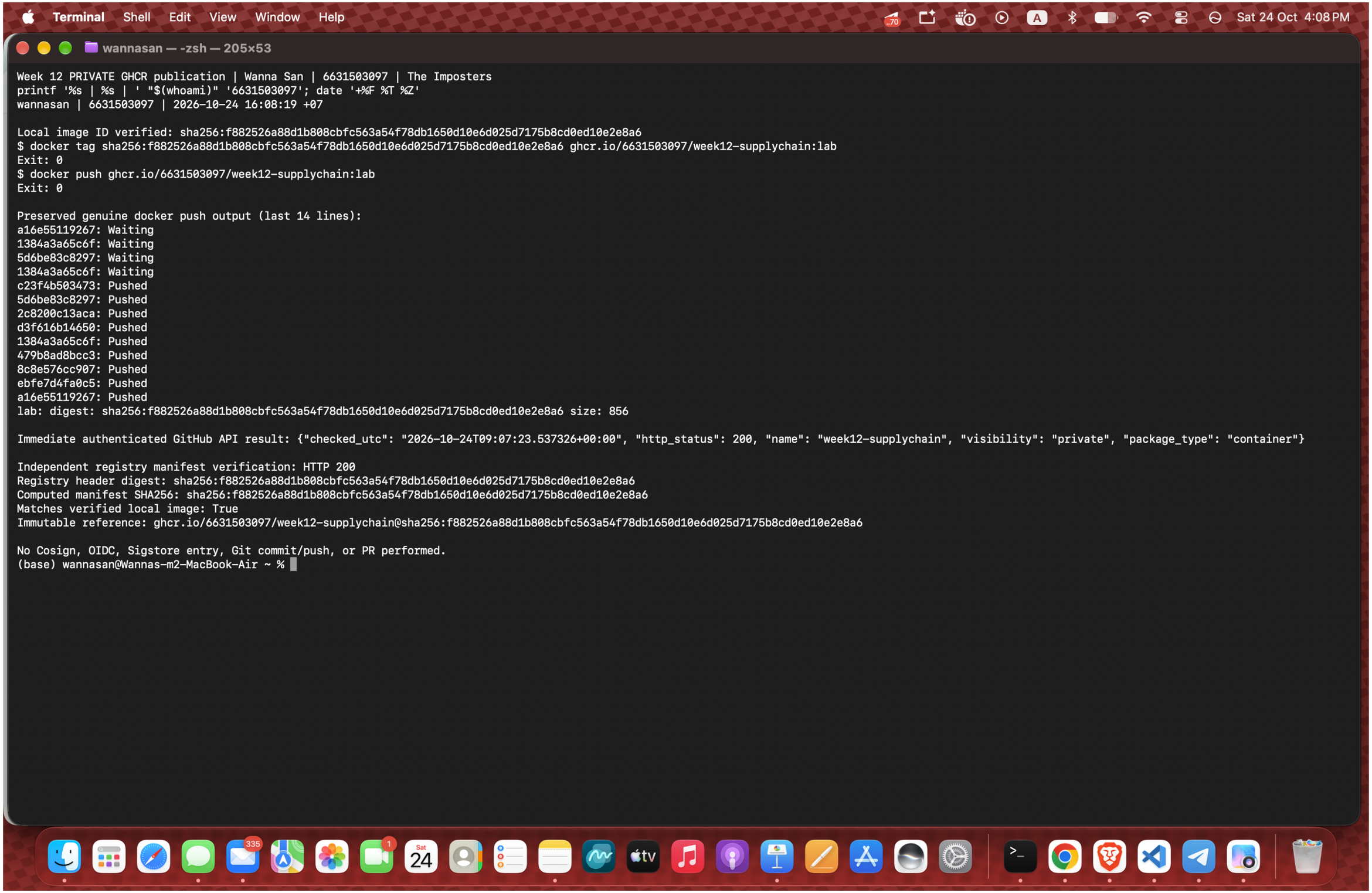
```
docker tag sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
ghcr.io/6631503097/week12-supplychain:lab
docker push ghcr.io/6631503097/week12-supplychain:lab
```

Both tag and push exited 0 after explicit publication-only approval. Immediately afterward the authenticated GitHub API returned visibility=private. The push digest, registry Docker-Content-Digest header, SHA-256 of returned manifest bytes and verified local image ID matched. No visibility change to public occurred. The registry index includes the ARM64 image and an existing BuildKit attestation descriptor; that alone is not an assessed SLSA level.

```
ghcr.io/6631503097/week12-supplychain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7
175b8cd0ed10e2e8a6
```

Package page (authentication may be required):
https://github.com/users/6631503097/packages/container/package/week12-supplychain . The private image may need explicit reviewer access; no access changes were made. Final private-visibility recheck is saved separately in evidence/final-qa/.

Task 3: genuine push output, immediate private visibility and independent digest verification. No token is displayed.



evidence/screenshots/task3-private-ghcr-publication.png | Original full-screen PNG; see source for full resolution.

6.3 Approved keyless signing and exact-policy verification

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign sign --yes --fulcio-auth-flow normal
--oidc-disable-ambient-providers --bundle /private/tmp/software-security-wk12/submission/we
ek12-6631503097/evidence/task3-signing/image.sigstore.json ghcr.io/6631503097/week12-supply
chain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
```

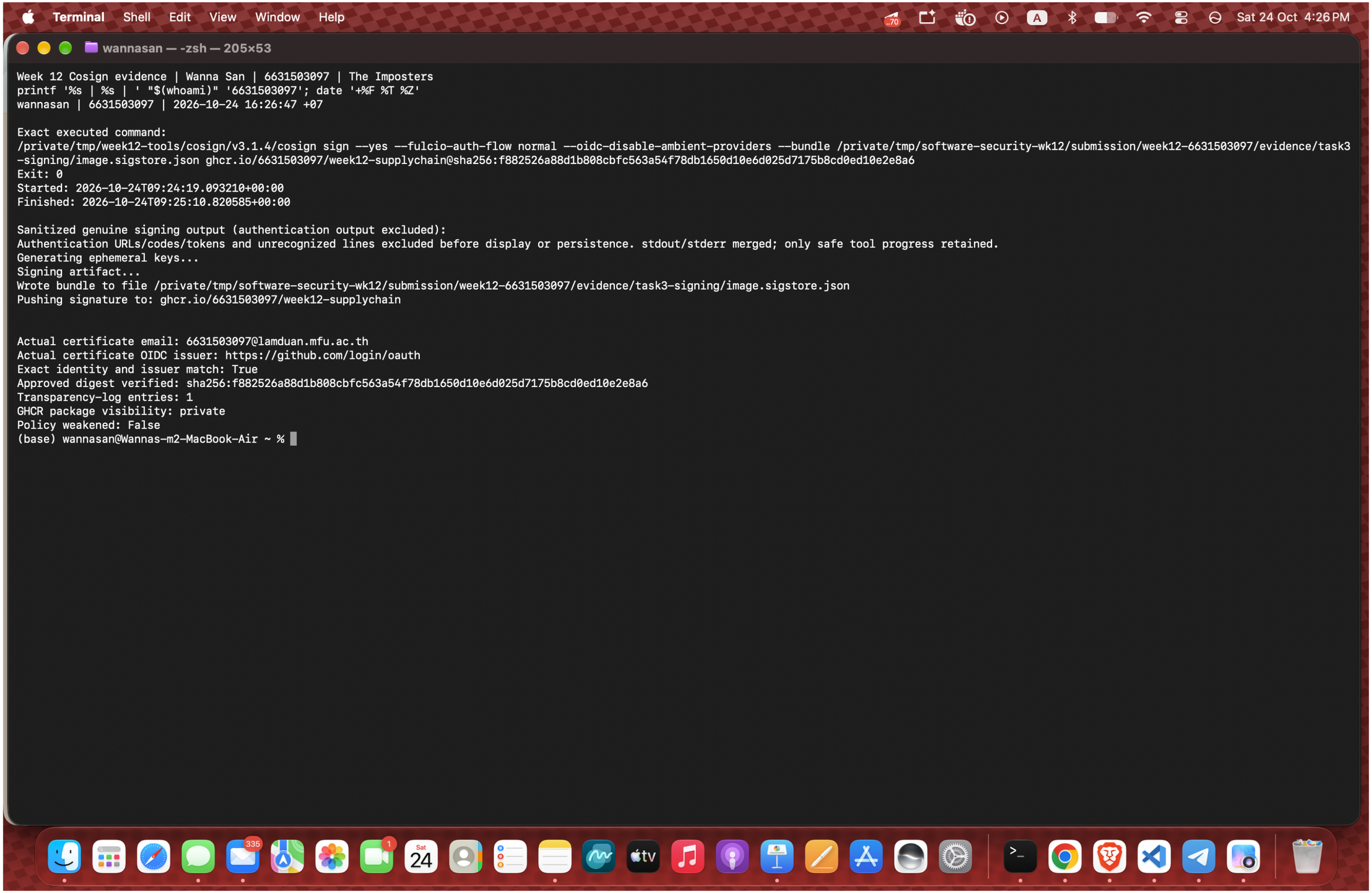
Official Cosign v3.1.4 exited 0. The student explicitly approved keyless signing and public metadata disclosure, then personally completed GitHub OIDC/2FA. --yes applied that existing consent; normal selected browser authentication and ambient providers were disabled. Only sanitized nonsecret progress was retained: ephemeral keys generated, artifact signed, bundle written, and signature pushed to the private GHCR repository. Raw auth URLs, codes and tokens were not stored.

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity
6631503097@lamduan.mfu.ac.th --certificate-oidc-issuer https://github.com/login/oauth ghcr.
io/6631503097/week12-supplychain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8
cd0ed10e2e8a6
```

Verification exited 0. The actual certificate SAN email was 6631503097@lamduan.mfu.ac.th and its OIDC issuer extension was https://github.com/login/oauth; both matched the exact expected policy. Cosign validated claims, trusted certificate evidence and transparency inclusion. No wildcard policy was substituted. The bundle image.sigstore.json contains one log entry, index 3438951395, with inclusion proof. Public disclosure was authorized even though the registry image remained private. [R9, R10]

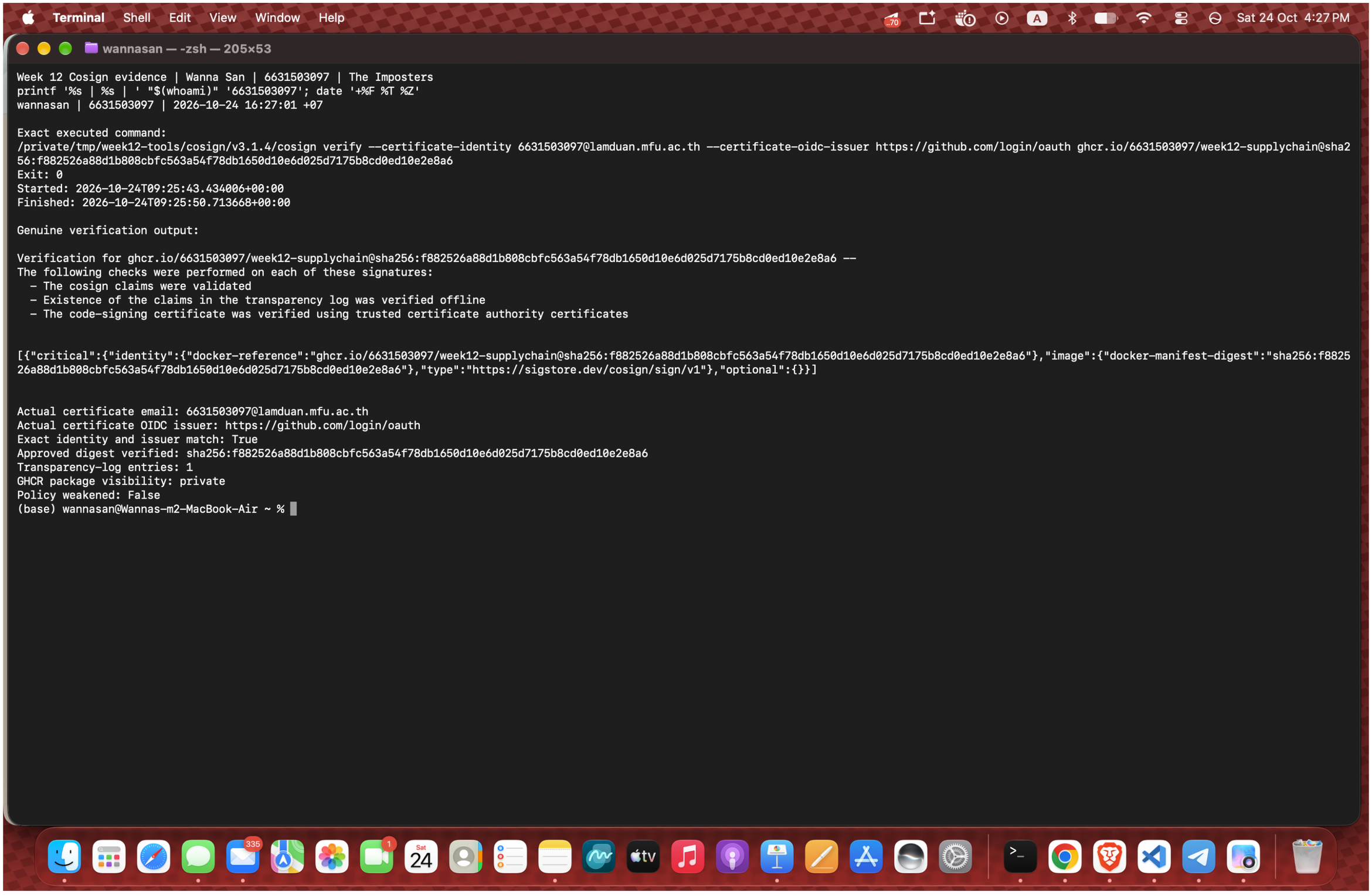
Remediation note (A08/CWE-829): require the authorized signer and immutable digest before a deployment decision; reject unknown identity, missing signatures and verification errors. A trusted signature identifies approval of those bytes, not absence of vulnerable or malicious dependencies.

Task 3: genuine sanitized signing transcript, exit 0, matching actual email/issuer and private GHCR status. Authentication screens are excluded.



evidence/screenshots/task3-cosign-signing.png | Original full-screen PNG; see source for full resolution.

Task 3: exact-policy verify command and success output for the approved digest; the certificate email and issuer match without regex relaxation.



evidence/screenshots/task3-cosign-verification.png | Original full-screen PNG; see source for full resolution.

6.4 Unsigned-image negative test

python:3.9-slim was checked, not assumed unsigned. Its manifest was accessible (HTTP 200); OCI referrers returned an empty manifests list (HTTP 200), legacy Cosign signature location returned 404, and cosign tree exited 0 reporting no related artifacts. The scope is absence of discoverable Cosign signatures for that exact index at check time, not every possible signing system or platform child.

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity-regexp '.*'
--certificate-oidc-issuer-regexp '.*' python:3.9-slim

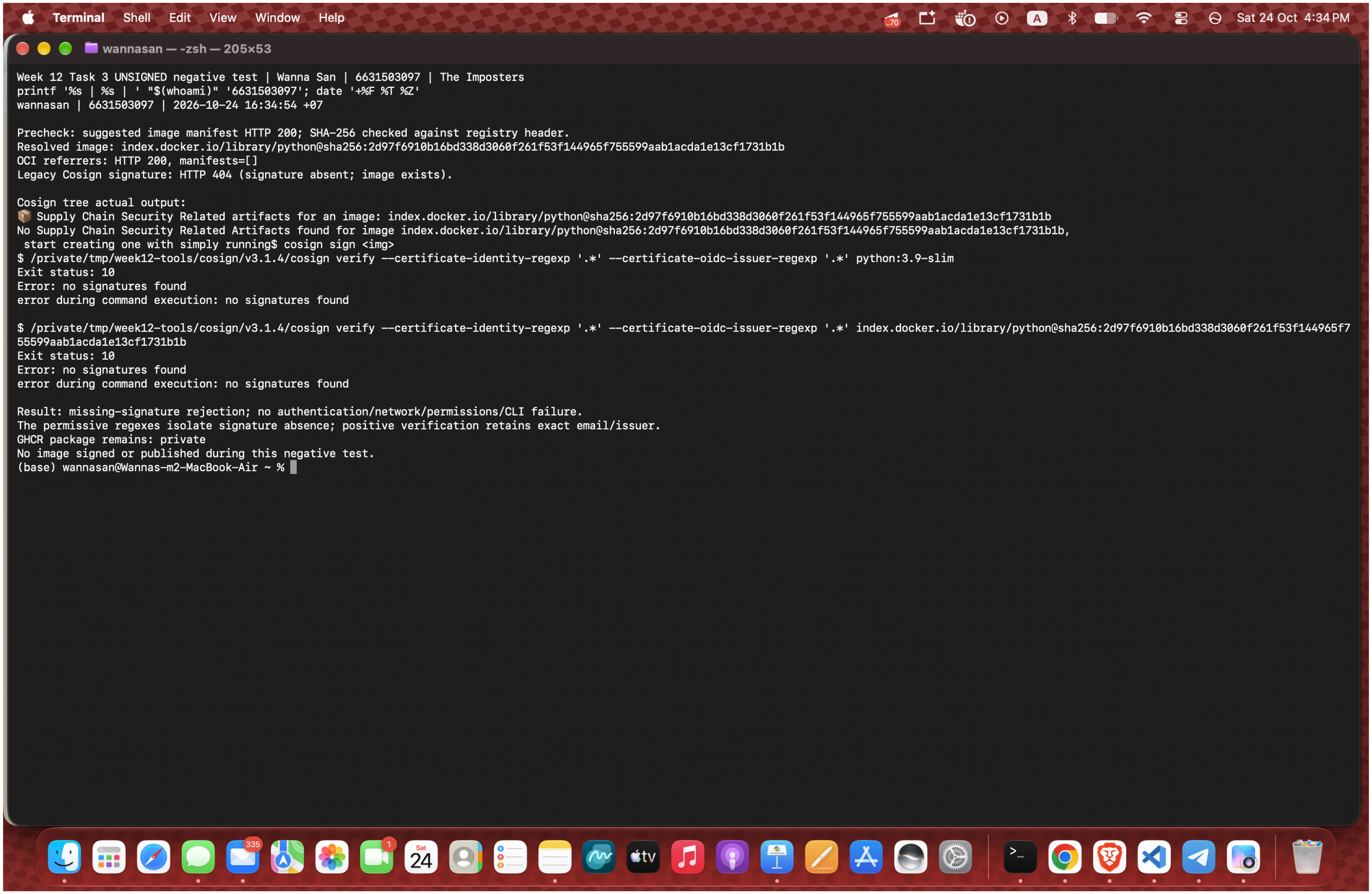
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity-regexp '.*'
--certificate-oidc-issuer-regexp '.*' index.docker.io/library/python@sha256:2d97f6910b16bd3
38d3060f261f53f144965f755599aab1acda1e13cf1731b1b
```

Both commands exited 10 with the genuine output below. Manifest/referrers access succeeded and both required identity flags were supplied, distinguishing absent signatures from network, permissions, authentication or CLI-usage failures. The permissive regexes are only the worksheet negative-test policy; production positive verification remains strict.

```
Error: no signatures found
error during command execution: no signatures found
```

The positive test succeeds because the approved digest has a valid signature from the expected identity/issuer with trusted proof. The negative image has no Cosign signature, so verification cannot establish that evidence and rejects it. This demonstrates the integrity boundary in A08/CWE-829; nothing was signed or published to manufacture a negative case.

Task 3: prechecked unsigned Python index, exact worksheet and digest-pinned commands, exit 10 and no signatures found. Genuine full-display identity evidence.



evidence/screenshots/task3-unsigned-negative.png | Original full-screen PNG; see source for full resolution.

7. Task 4 - defenses and pre-deploy gate

7.1 Pin + hashes and a single trusted index

```
/private/tmp/week12-task4-hash-20261024/bin/pip install --require-hashes -r
requirements.txt

/private/tmp/week12-task4-hash-20261024/bin/pip install --require-hashes --index-url
https://pypi.org/simple -r requirements.txt
```

Both ran against the unchanged lab manifest in a disposable Python 3.13 venv with pip 25.0; both exited 1 specifically because hashes were absent. PIP_CONFIG_FILE=/dev/null and a clean subprocess environment excluded inherited extra-index settings. The venv retained only pip; no vulnerable lab dependencies were installed into the main environment.

```
ERROR: Hashes are required in --require-hashes mode, but they are missing from some
requirements.
Flask==1.1.4 --hash=sha256:c34f04500f2cbbea882b1acb02002ad6fe6b7ffa64a6164577995657f50aed22
```

This is a genuine refusal after downloading the Flask wheel to compute its hash. It does not produce a fully hashed lockfile or demonstrate an actual malicious replacement. An approved lock must cover all resolved dependencies and permitted artifacts using reviewed hashes; copying a hash from an untrusted download alone does not establish trust.

PyPI was the single source for this public-dependency demonstration, not a claimed private internal registry. A real team should use one controlled private/curated proxy containing internal artifacts and vetted public dependencies. Merging sources with --extra-index-url permits competing candidates; a single index works only when alternate config, environment, find-links and direct references are controlled. The existing Task 2 images show the same pair changing from public 99.0.0 to private 1.4.0. [R1]

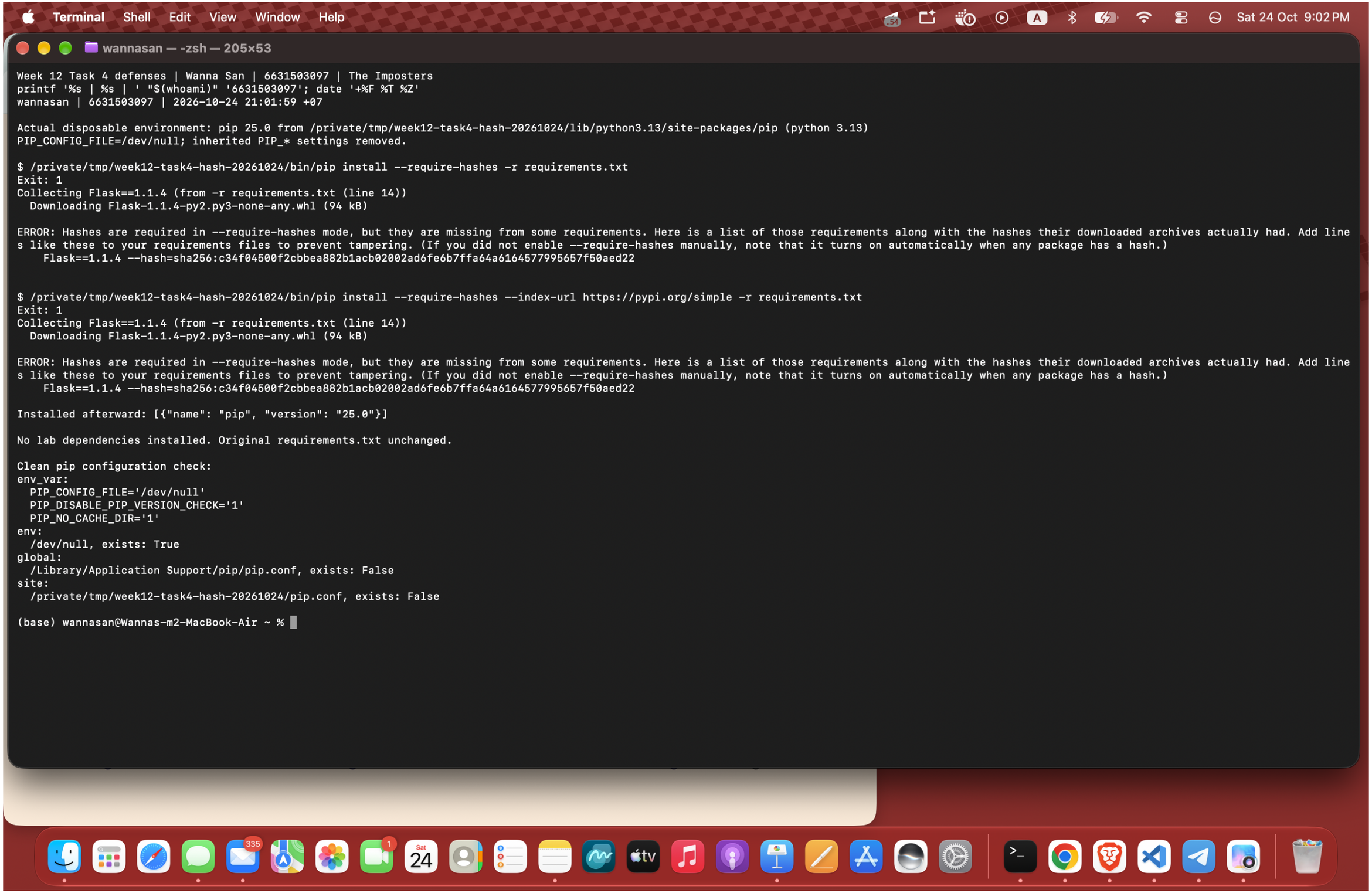
7.2 Namespace scoping and the strongest defense

Practical proposal: define an organization-specific internal naming convention, enforce allowed package names and access controls in a private registry, and use one curated index. Reserving a public name where policy permits can reduce collisions, but a Python prefix is not enforceable namespace ownership or routing. No name was reserved or package published in this exercise.

Student-confirmed choice: single curated trusted index. Suggested rationale - review before adopting: it removes the competing public source at the point of candidate selection and centralizes policy, directly matching the simulation before/after result. It is not sufficient alone: registry compromise and maintainer compromise remain, so reviewed hash locks, SCA and the signed-digest gate are complementary.

Remediation note (A03/A08; CWE-1357/CWE-829): enforce one vetted source and reviewed hash locks to reduce substitution and unapproved bytes. Keep SCA and compatibility testing because integrity controls do not fix the known CVEs (CWE-1395).

Task 4: both actual missing-hash refusals, printed Flask SHA-256 and clean pip configuration, with exact identity command. Only pip remains installed.



evidence/screenshots/task4-hashes-single-index.png | Original full-screen PNG; see source for full resolution.

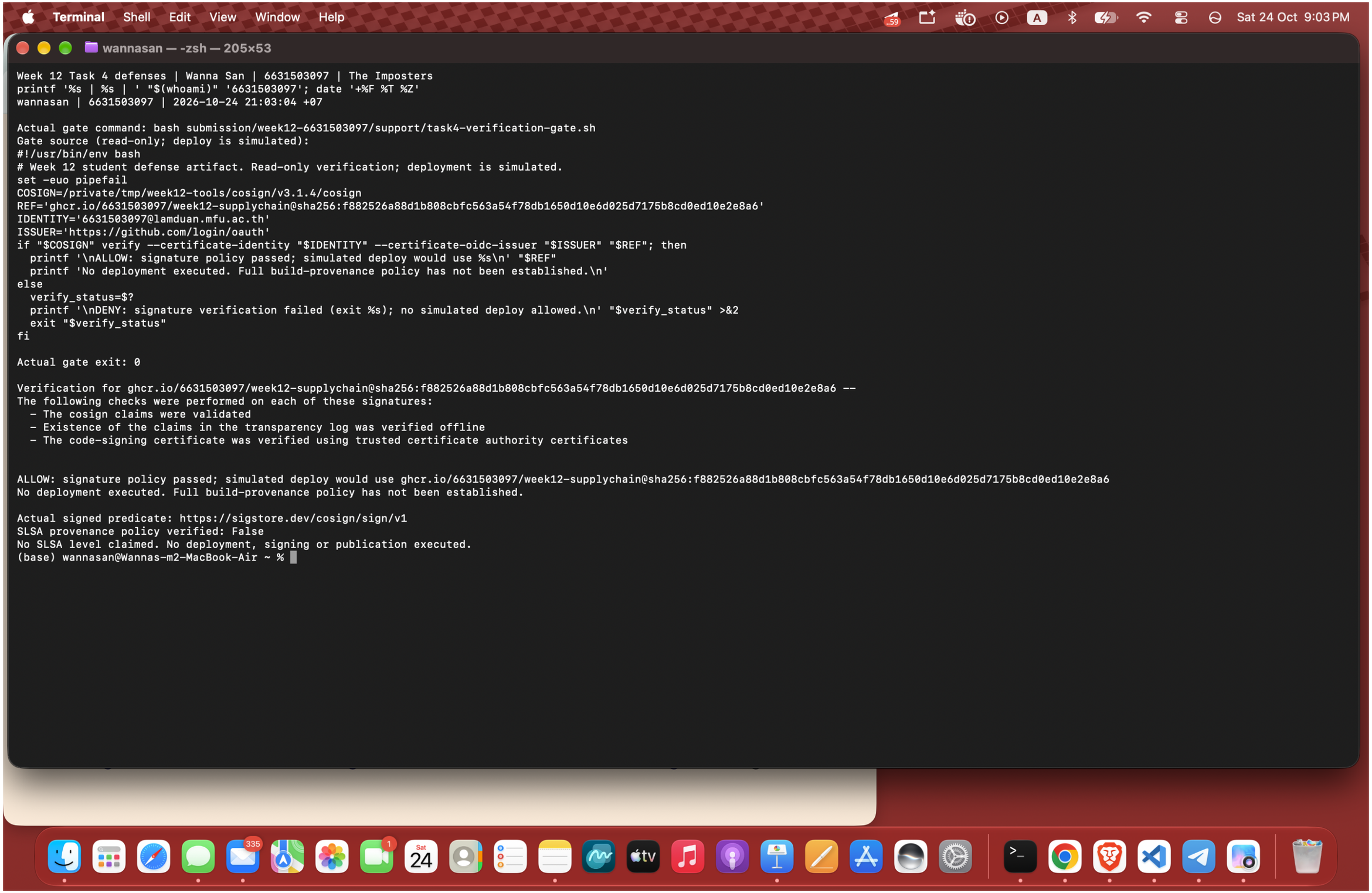
7.3 Practical verification gate; deployment simulated

```
#!/usr/bin/env bash
# Week 12 student defense artifact. Read-only verification; deployment is simulated.
set -euo pipefail
COSIGN=/private/tmp/week12-tools/cosign/v3.1.4/cosign
REF='ghcr.io/6631503097/week12-supplychain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d
025d7175b8cd0ed10e2e8a6'
IDENTITY='6631503097@lamduan.mfu.ac.th'
ISSUER='https://github.com/login/oauth'
if "$COSIGN" verify --certificate-identity "$IDENTITY" --certificate-oidc-issuer "$ISSUER"
"$REF"; then
  printf '\nALLOW: signature policy passed; simulated deploy would use %s\n' "$REF"
  printf 'No deployment executed. Full build-provenance policy has not been established.\n'
else
  verify_status=$?
  printf '\nDENY: signature verification failed (exit %s); no simulated deploy allowed.\n'
"$verify_status" >&2
  exit "$verify_status"
fi
```

Actual command: bash submission/week12-6631503097/support/task4-verification-gate.sh . It exited 0 and printed ALLOW only after exact-policy verification. No deployment was executed. In a real pipeline, prevent bypass and deploy the same verified digest; all verification, auth and network errors must deny. Logs are in evidence/task4/gate.*; the code is the local defense commit 336aaf8f9457d0a573c1649ff83e4cb0c2c99d2b.

This gate verifies a signature, not the full SLSA build-provenance policy implied by the worksheet heading. The observed signed predicate is https://sigstore.dev/cosign/sign/v1 with an empty predicate. A full provenance gate would additionally authenticate a trusted builder and check the subject digest, source/revision, build type/parameters and relevant inputs; no such complete policy or SLSA level is claimed. [R8]

Task 4: unchanged student gate source, actual read-only verification PASS and simulated ALLOW, with explicit provenance limitation and identity proof.



evidence/screenshots/task4-signature-gate-confirmed.png | Original full-screen PNG; see source for full resolution.

8. Part 4 - security analysis and reflection

8.1 Required finding/tool/mapping/fix table

Finding	Tool / evidence	OWASP / CWE	Fix / limitation
Known vulnerable packages	sca_scan.sh, saved Trivy/pip-audit	A03 / CWE-1395; CWE-1104 only if unmaintained	Compatible patched dependency set; proposed, not implemented
Confused source selection	Resolver simulation	A03 / CWE-1357, CWE-829	Single curated index plus naming/access policy
Unhashed requirement	pip --require-hashes refusal	A03/A08 / CWE-1357, CWE-829	Complete reviewed hash lock; refusal only demonstrated
Unsigned or unexpected signer	Cosign positive/negative and gate	A08 / CWE-829	Enforce digest, exact identity/issuer and failure handling
Reusable signing-key exposure	Keyless Fulcio/Rekor flow	A08 / CWE-321 risk reduction	Ephemeral signing key; protect the identity account

8.2 XZ Utils backdoor - CVE-2024-3094

Andres Freund reported the upstream xz/liblzma backdoor on 29 March 2024 after investigating abnormal SSH behavior. The affected release tarballs included 5.6.0 and 5.6.1; part of the injected build logic was in release tarballs rather than the corresponding upstream file, illustrating why inspecting only a repository snapshot can miss release-time manipulation. The malicious build path conditionally modified liblzma output. This is analyzed here as A03 supplier/build compromise and A08 artifact-integrity failure, with CWE-829/CWE-1357 as reasoned classroom mappings, not a claim that these are the CVE record's official assigned CWE. [R11, R2, R4]

An SBOM can help find affected xz versions but does not identify malicious intent by itself. A valid upstream signature may faithfully authenticate a malicious release made through a compromised trusted maintainer. A provenance gate helps only when it verifies meaningful trusted source/build expectations and input relationships; matching an approved signer alone would not necessarily catch this backdoor. Independent release review, comparison of generated tarballs to source, controlled builds and behavioral investigation provide additional evidence. These are analysis and recommendations, not tests performed against XZ in this lab. [R11, R8]

8.3 Suggested mitigation and SLSA self-assessment - student review required

Suggested reflection, not a statement of private experience: The evidence demonstrates a component inventory, an identity-bound signature, unsigned-image rejection, and a signature-gated simulated deployment. It does not establish a SLSA level because the BuildKit attestation was not fully assessed and no trusted build-platform provenance policy was verified. The student chose a single curated trusted index as the strongest defense; the proposed reason is that it removes competing public candidates before installation, while hashes and verified release digests address different integrity boundaries. Review and rewrite this paragraph in your own words before submitting. [R7, R8]

8.4 Evidence & Integrity: suggested own-words explanations

Suggested answer to "What did you do and why did the vulnerability work?": The controlled resolver simulation pooled private and public candidates and selected a higher public version. The SCA scans separately showed known advisories in the intentionally old dependency tree; they did not demonstrate that every endpoint was exploitable.

Suggested answer to "Why does the fix stop it and what could still break it?": One curated source excludes the competing index, hashes constrain acceptable bytes, and exact-identity verification requires approval of the same

immutable digest. Compromised trusted sources/accounts, unreviewed hashes, gate bypass or malicious-but-authorized dependencies can still defeat those assumptions. These explanations are AI-assisted suggestions awaiting student adoption.

9. Audit the AI - actual response, critique and verified correction

The actual Task 4 prompt and full assistant response from this conversation are preserved verbatim in ai/task4-prompt-verbatim.txt and ai/task4-response-verbatim.md. They were saved before this critique was composed. This is an available real interaction, not an invented model output. The complete response follows, including its historical status and original links; no later claims are attributed to that earlier response.

```
**Task 4's technical exercises are complete.**

- **Hash enforcement:** pip refused the original unhashed requirements with exit **1** and
printed Flask's actual SHA-256. The disposable environment contains only pip; no vulnerable
 dependencies were installed.
- **Single index:** demonstrated `--index-url https://pypi.org/simple` with inherited pip
settings removed. The report links this to Task 2's verified before/after evidence.
- **Verification gate:** the signed, digest-pinned image passed the exact email/issuer
policy, exit **0**, allowing only a **simulated** deployment.
- **SLSA:** signature verification is distinguished from build-provenance verification.
**No SLSA level is claimed.**

Saved:

- [Commands, outputs, exit codes and
evidence](/private/tmp/software-security-wk12/submission/week12-6631503097/evidence/task4)
- [Hash/index screenshot](/private/tmp/software-security-wk12/submission/week12-6631503097/
evidence/screenshots/task4-hashes-single-index.png)
- [Gate screenshot](/private/tmp/software-security-wk12/submission/week12-6631503097/eviden
ce/screenshots/task4-signature-gate-confirmed.png)
- [Gate script](/private/tmp/software-security-wk12/submission/week12-6631503097/support/ta
sk4-verification-gate.sh)
- [Updated report](/private/tmp/software-security-wk12/submission/week12-6631503097/week12-
tasks0-2.md)

Remaining written work: personalize the namespace-scoping explanation, explain the
simulation's before/after result, and choose the **most effective defense and why**.
Reflection questions remain separate.

No instructor or `wk06` changes, additional publication/signing, Git commits/pushes, PR, or
 PDF generation occurred.
```

9.1 Suggested critique for student review

Exact line under review: "Verification gate: the signed, digest-pinned image passed the exact email/issuer policy, exit 0, allowing only a simulated deployment." This is factually supported, but an incomplete security-verification story: a positive result alone does not demonstrate the gate denies an unauthorized signer. The original technical report explicitly left that branch unexercised; the full response did not provide a negative gate result. This is a missed verification edge case, not a fabricated error in otherwise correct statements.

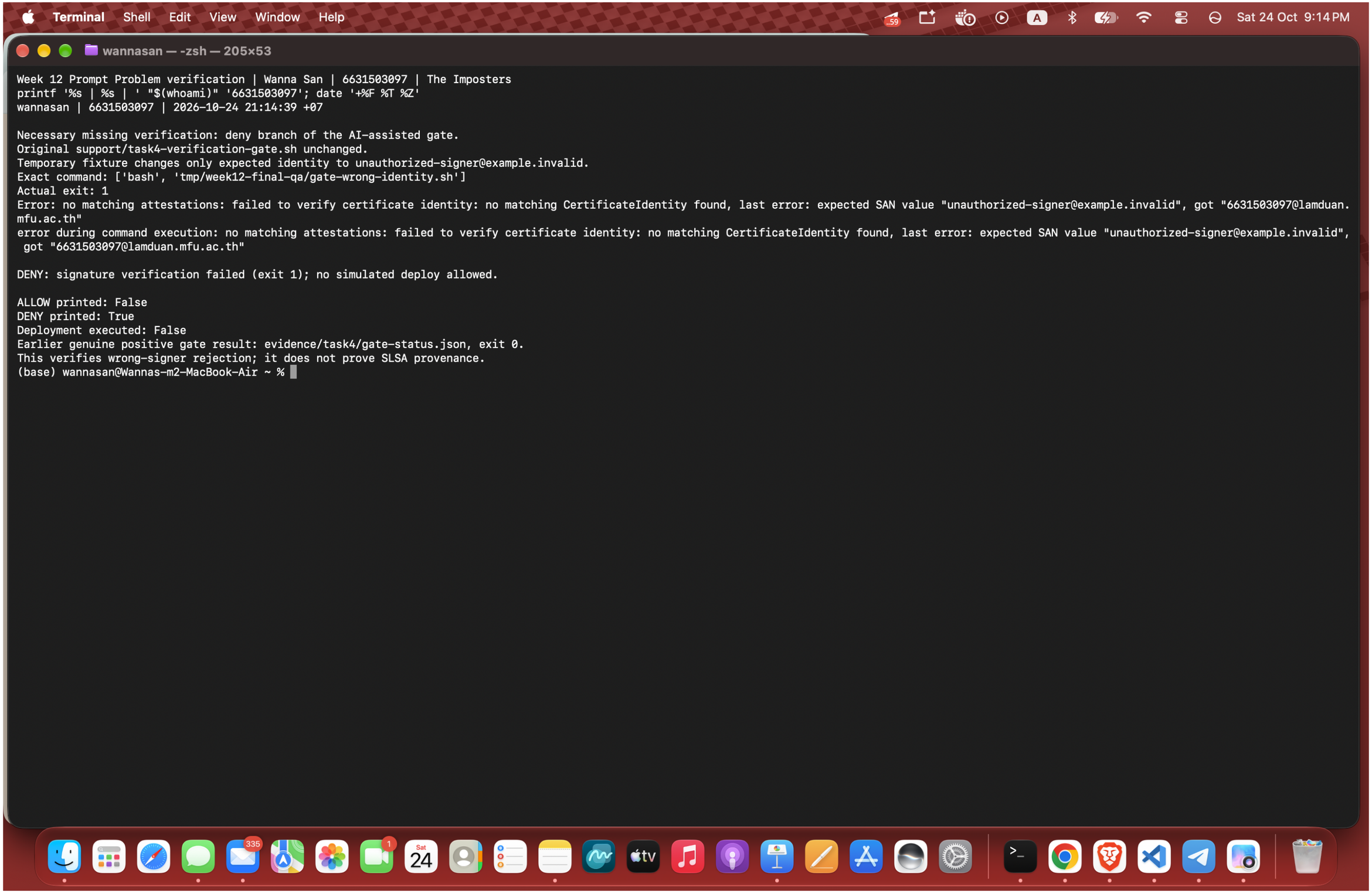
A second limitation is the exact line "Single index: demonstrated --index-url https://pypi.org/simple with inherited pip settings removed." It accurately describes the command but is not proof that a private registry was configured or that a compromised allowed source is safe. The correct interpretation is a clean single-source demonstration plus the controlled resolver evidence. pip documents combined-source risks; SLSA distinguishes signature authenticity from verified build provenance. [R1, R8]

9.2 Actual additional verification and corrected account

To close the missing check, Codex ran a temporary copy of the existing gate that changed only the expected email to unauthorized-signer@example.invalid. The original defense script was unchanged. Actual command: bash tmp/week12-final-qa/gate-wrong-identity.sh . It exited 1 with an explicit certificate-identity mismatch, printed DENY and never printed ALLOW. No deployment, signing or publication occurred. The necessary new result is in evidence/final-qa/gate-wrong-identity-*; the earlier actual success result remains in evidence/task4/.

Suggested 3-sentence correction - student review required: A passing signature check establishes the expected signer for this digest, but does not by itself prove denial behavior or trustworthy build provenance. The same gate now has a genuine wrong-identity rejection, while the original unsigned-image test separately proves missing-signature rejection. This strengthens the observed signature-policy evidence without claiming a full lockfile, a private-index implementation or a SLSA level.

Audit the AI / Prompt Problem: genuine newly necessary wrong-signer gate test; exit 1, DENY true, ALLOW false, no deployment. Exact identity stamp visible.



evidence/screenshots/prompt-wrong-signer-rejection.png | Original full-screen PNG; see source for full resolution.

10. Comprehension & Prompt

10.1 Explain in Plain English (EiPE) - suggested answer

The small Flask app returns simple JSON at / and /health, but its image brings in an intentionally old dependency tree. The exercise shows how software can inherit known flaws or choose the wrong package before the app itself runs. The simulation demonstrates source confusion; the scans show dependency advisories rather than a demonstrated exploit of these endpoints. [Student review: rewrite these three sentences in your own words.]

10.2 Prompt Problem - actual final prompt and verification

The final prompt used for this exercise is the real Task 4 user request reproduced below; the response is the full actual answer in Section 9. It is broad, but explicitly requires isolated hash enforcement, a digest-pinned identity policy and no unsupported SLSA claim. No shorter imagined prompt or invented refinement is substituted. The output defense artifact is support/task4-verification-gate.sh; no automated claim that the vulnerability was patched everywhere is made.

```
Continue Week 12 Task 4 in the isolated worktree:

/private/tmp/software-security-wk12

Tasks 0-3 are complete, with verified evidence.

Please complete the technical defense exercises for Task 4 using the instructor's Week 12
worksheet and existing project files.

1. Inspect the exact Task 4 requirements before running commands.
2. Demonstrate pip hash enforcement using `pip install --require-hashes -r
requirements.txt` in an isolated, disposable environment. Capture the genuine refusal
caused by missing hashes.
3. Demonstrate the safer single-trusted-index configuration using `--index-url`, and
explain why combining private and public indexes with `--extra-index-url` can create
dependency-confusion risks.
4. Document a practical provenance verification gate using our signed, digest-pinned GHCR
image and the exact Cosign certificate identity and issuer.
5. Distinguish signature verification from full SLSA provenance attestation. Do not claim a
 SLSA level without supporting evidence.
6. Capture exact commands, outputs, exit codes, and genuine full-screen screenshots showing
 the required student identity command.
7. Save evidence under the existing Week 12 submission directory.
8. Update the report with concise explanations and OWASP/CWE mappings.
9. Identify any remaining Task 4 requirements that need written analysis rather than
command execution.

Do not change instructor files or the original wk06 worktree.
Do not install vulnerable dependencies into my main environment.
Do not publish, sign, or push any additional artifacts.
Do not commit, push Git code, create a PR, or generate the final PDF yet.

Stop after the technical exercises and report the results and remaining written
requirements.
```

Verification method actually performed: (1) reuse the recorded original gate PASS, exit 0; (2) inspect the original hardcoded digest/email/issuer and failure-exit handling; (3) run only the newly necessary wrong-email fixture, which exited 1 with a certificate-identity mismatch and no ALLOW. The proposed unauthorized-signer acceptance fails under that policy. This is a verification of signer-policy enforcement, not a live dependency-confusion exploit or complete SLSA gate. Student review of the prompt/critique and ability to explain the evidence are still required.

11. Conclusion and GitHub/course submission

The lab produced genuine SCA evidence, a controlled dependency-confusion comparison, an image-derived CycloneDX inventory, a private published image, a matching-identity signature, missing-signature rejection and a simulated pre-deploy gate. Hash enforcement refused the original unhashed file. These are complementary trust controls, with explicitly recorded limits: no production deployment, no full hashed lockfile, no package-version remediation and no supported SLSA-level claim.

Local defense commit: 336aaf8f9457d0a573c1649ff83e4cb0c2c99d2b. Prospective commit link (not remotely resolvable until a separately approved push): https://github.com/6631503097/software-security/commit/336aaf8f9457d0a573c1649ff83e4cb0c2c99d2b . Repository: https://github.com/6631503097/software-security . Branch: wk12. PR URL: NOT CREATED - explicit push/PR approval required. This PDF cannot truthfully claim a submitted GitHub code fix yet.

Course actions for the student: review/adopt the labeled personal text; obtain the final approved GitHub commit/PR links after publication; upload Wk12_6631503097.pdf to https://learn.zcr.ai/submit using your issued submission code. Complete the individual one-attempt quiz at https://learn.zcr.ai/quiz using the issued code; no quiz attempt or portal upload was performed by Codex. The repository portal implementation (labs/live-quiz/submission.py) sets a 20 MiB per-file limit; the PDF is checked below that limit without changing screenshot pixels. The repository deadline rule is before the next class unless the instructor says otherwise; the exact class date was not inferred. Quizzes are individual and irreversible between questions.

Requirement / rubric	Status
Lecture questions (20%)	Five sourced answers supplied
Tasks 0-3 and evidence (40%)	Observed results, commands, status and screenshots supplied
Task 4 defense (25%)	Technical evidence and code gate supplied; rationale draft needs student review
Reflection (15%)	Mapping, XZ analysis and cautious self-assessment supplied as labeled suggested text
Audit the AI / EiPE / Prompt	Actual interaction preserved; technical verification complete; student critique/own-words adoption pending
Identity screenshots	15 accepted originals embedded; rejected captures excluded
GitHub code fix	Local defense commit exists; branch push/PR not authorized or performed
PDF / quiz / portal	PDF generated and checked; quiz status unknown; portal submission pending

Submission readiness: technically assembled, but not fully submitted or ready to represent as entirely student-authored. Personal-text review, separately approved GitHub publication/PR, and the student portal/quiz actions remain. No personal opinion is fabricated; the chosen strongest defense is user-confirmed and the reason is explicitly suggested.

12. References

[R1] pip install - resolver and index behavior. https://pip.pypa.io/en/stable/cli/pip_install/

[R2] OWASP A03:2025 Software Supply Chain Failures. https://top10.owasp.org/2025/A03_2025-Software_Supply_Chain_Failures/

[R3] MITRE CWE-1395: Dependency on Vulnerable Third-Party Component. https://cwe.mitre.org/data/definitions/1395.html

[R4] MITRE CWE-829 and CWE-1357. https://cwe.mitre.org/data/definitions/829.html

[R4b] MITRE CWE-1357: Reliance on Insufficiently Trustworthy Component. https://cwe.mitre.org/data/definitions/1357.html

[R5] CycloneDX specification overview. https://cyclonedx.org/specification/overview/

[R6] SPDX overview. https://spdx.dev/learn/overview/

[R7] SLSA v1.2 Build: Track Basics. https://slsa.dev/spec/v1.2/build-track-basics

[R8] SLSA v1.2 Build: Provenance. https://slsa.dev/spec/v1.2/build-provenance

[R9] Sigstore OIDC usage in Fulcio. https://docs.sigstore.dev/certificate_authority/oidc-in-fulcio/

[R10] Sigstore privacy and identity disclosure. https://blog.sigstore.dev/privacy-in-sigstore-57cac15af0d0/

[R11] Andres Freund, original xz/liblzma backdoor report, 29 March 2024. https://openwall.com/lists/oss-security/2024/03/29/4

[R12] OWASP A08:2025 Software or Data Integrity Failures. https://top10.owasp.org/2025/A08_2025-Software_or_Data_Integrity_Failures/

[R13] GitHub Container registry authentication and private default. https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry

[R14] Sigstore OIDC verification policy cheat sheet. https://docs.sigstore.dev/quickstart/verification-cheat-sheet/

Primary course sources: labs/week12-supply-chain/worksheet.md, README.md, dependency-confusion.md, requirements.txt, Dockerfile, app.py, sca_scan.sh, sign.sh; slides/week12.md; root SUBMISSION.md and ETHICS.md. All instructor sources remained unchanged. External references were checked on 24 October 2026. Source/certificate identity and the student number are intentionally included; no authentication secrets are included.

Appendix A - authoritative command and evidence index

All paths below are relative to submission/week12-6631503097/. Original PNGs are embedded without pixel editing; captions are outside the image. Full logs contain details omitted from concise excerpts. JSON commands use the exact executed argument list; wrapper-added timeouts and status behavior are documented in their source.

evidence/task1/process-status.jsonl

```
/usr/local/bin/docker run --rm -v
/private/tmp/software-security-wk12/labs/week12-supply-chain:/src aquasec/trivy:latest fs
--scanners vuln --severity HIGH,CRITICAL /src
```

Actual exit: 0

```
/usr/local/bin/docker run --rm -v
/private/tmp/software-security-wk12/labs/week12-supply-chain:/src -w /src python:3.9-slim
sh -c 'pip install --quiet pip-audit && pip-audit -r requirements.txt'
```

Actual exit: 1

```
/usr/local/bin/docker image inspect week12-supplychain:lab
```

Actual exit: 1

evidence/task3/build-status.json

```
docker build --progress=plain -t week12-supplychain -t week12-supplychain:lab .
```

Actual exit: 0

evidence/task3/sbom-status.json

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock -v
/private/tmp/software-security-wk12/submission/week12-6631503097/evidence/task3:/evidence
aquasec/trivy:latest image --image-src docker --format cyclonedx --output
/evidence/sbom.cdx.json
sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
```

Actual exit: 0

evidence/task3/validation-status.json

```
/private/tmp/week12-sbom-validation/bin/python
submission/week12-6631503097/support/validate-sbom.py
```

Actual exit: 0

evidence/task3-publication/push-status.json

```
docker push ghcr.io/6631503097/week12-supplychain:lab
```

Actual exit: 0

evidence/task3-signing/sign-status.json

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign sign --yes --fulcio-auth-flow normal
--oidc-disable-ambient-providers --bundle /private/tmp/software-security-wk12/submission/we
ek12-6631503097/evidence/task3-signing/image.sigstore.json ghcr.io/6631503097/week12-supply
chain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8cd0ed10e2e8a6
```

Actual exit: 0

evidence/task3-signing/verify-status.json

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity
6631503097@lamduan.mfu.ac.th --certificate-oidc-issuer https://github.com/login/oauth ghcr.
io/6631503097/week12-supplychain@sha256:f882526a88d1b808cbfc563a54f78db1650d10e6d025d7175b8
cd0ed10e2e8a6
```

Actual exit: 0

evidence/task3-negative/verify-worksheet-status.json

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity-regexp '.*'
--certificate-oidc-issuer-regexp '.*' python:3.9-slim
```

Actual exit: 10

evidence/task3-negative/verify-pinned-status.json

```
/private/tmp/week12-tools/cosign/v3.1.4/cosign verify --certificate-identity-regexp '.*'
--certificate-oidc-issuer-regexp '.*' index.docker.io/library/python@sha256:2d97f6910b16bd3
38d3060f261f53f144965f755599aab1acda1e13cf1731b1b
```

Actual exit: 10

evidence/task4/hash-refusal-status.json

```
/private/tmp/week12-task4-hash-20261024/bin/pip install --require-hashes -r
requirements.txt
```

Actual exit: 1

evidence/task4/single-index-refusal-status.json

```
/private/tmp/week12-task4-hash-20261024/bin/pip install --require-hashes --index-url
https://pypi.org/simple -r requirements.txt
```

Actual exit: 1

evidence/task4/gate-status.json

```
bash submission/week12-6631503097/support/task4-verification-gate.sh
```

Actual exit: 0

evidence/final-qa/gate-wrong-identity-status.json

```
bash tmp/week12-final-qa/gate-wrong-identity.sh
```

Actual exit: 1

Artifact paths and hashes are indexed in evidence/final-qa/. The production gate is unchanged.